

Design Patterns & Principles

Assignment – Complete Answers (Java)

Exercise 1: Singleton Pattern – Logger

The **Singleton Pattern** ensures a class has only one instance and provides a global access point to it. It is used here for a Logger so that all parts of the application share one consistent logging channel.

Implementation

```
public class Logger {

    // 1. Private static instance
    private static Logger instance;

    // 2. Private constructor - prevents direct instantiation
    private Logger() {
        System.out.println("Logger initialized.");
    }

    // 3. Public static accessor (thread-safe via synchronized)
    public static synchronized Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}

// Test class
public class SingletonTest {
    public static void main(String[] args) {
        Logger l1 = Logger.getInstance();
        Logger l2 = Logger.getInstance();

        l1.log("Application started.");
        l2.log("User logged in.");

        // Verify same instance
        System.out.println("Same instance: " + (l1 == l2)); // true
    }
}
```

Key Points

- Private constructor prevents *new Logger()*.

- Static field holds the single instance across all calls.
- *synchronized* keyword makes it thread-safe.

- For better performance use **Double-Checked Locking** or an **Enum Singleton**.

Exercise 2: Factory Method Pattern – Document Management

The **Factory Method Pattern** defines an interface for creating objects but lets subclasses decide which class to instantiate. This decouples the client from concrete product classes.

Implementation

```
// Product interface
interface Document {
    void open();
}

// Concrete products
class WordDocument implements Document {
    public void open() { System.out.println("Opening Word document."); }
}
class PdfDocument implements Document {
    public void open() { System.out.println("Opening PDF document."); }
}
class ExcelDocument implements Document {
    public void open() { System.out.println("Opening Excel document."); }
}

// Abstract factory
abstract class DocumentFactory {
    public abstract Document createDocument();

    public void openDocument() { // template method
        Document doc = createDocument();
        doc.open();
    }
}

// Concrete factories
class WordFactory extends DocumentFactory {
    public Document createDocument() { return new WordDocument(); }
}
class PdfFactory extends DocumentFactory {
    public Document createDocument() { return new PdfDocument(); }
}
class ExcelFactory extends DocumentFactory {
    public Document createDocument() { return new ExcelDocument(); }
}

// Test
public class FactoryTest {
    public static void main(String[] args) {
        DocumentFactory factory = new PdfFactory();
        factory.openDocument(); // Opening PDF document.

        factory = new WordFactory();
        factory.openDocument(); // Opening Word document.
    }
}
```

Key Points

- Client code works with the *DocumentFactory* abstraction, not concrete classes.
- Adding a new document type (e.g., CSV) only requires a new factory subclass.
- Follows the **Open/Closed Principle** – open for extension, closed for modification.

Exercise 3: Builder Pattern – Computer

The **Builder Pattern** separates the construction of a complex object from its representation, allowing the same process to create different configurations.

Implementation

```
public class Computer {
    private String cpu;
    private String ram;
    private String storage;
    private String gpu;
    private boolean hasBluetooth;

    // Private constructor - only Builder can instantiate
    private Computer(Builder b) {
        this.cpu = b.cpu;
        this.ram = b.ram;
        this.storage = b.storage;
        this.gpu = b.gpu;
        this.hasBluetooth = b.hasBluetooth;
    }

    @Override
    public String toString() {
        return "Computer{cpu=" + cpu + ", ram=" + ram +
            ", storage=" + storage + ", gpu=" + gpu +
            ", bluetooth=" + hasBluetooth + "}";
    }

    // Static nested Builder class
    public static class Builder {
        private String cpu;
        private String ram;
        private String storage;
        private String gpu = "Integrated";
        private boolean hasBluetooth = false;

        public Builder cpu(String cpu) { this.cpu = cpu; return this; }
        public Builder ram(String ram) { this.ram = ram; return this; }
        public Builder storage(String s) { this.storage = s; return this; }
        public Builder gpu(String gpu) { this.gpu = gpu; return this; }
        public Builder bluetooth(boolean bt) { this.hasBluetooth = bt; return this; }

        public Computer build() { return new Computer(this); }
    }
}

// Test
```

```

public class BuilderTest {
    public static void main(String[] args) {
        Computer gaming = new Computer.Builder()
            .cpu("Intel i9").ram("32GB")
            .storage("1TB SSD").gpu("RTX 4090")
            .bluetooth(true).build();

        Computer office = new Computer.Builder()
            .cpu("Intel i5").ram("8GB")
            .storage("512GB HDD").build();

        System.out.println(gaming);
        System.out.println(office);
    }
}

```

Key Points

- Avoids telescoping constructors with many optional parameters.
- Builder's fluent API (method chaining) improves readability.
- Immutable objects can be created safely.

Exercise 4: Adapter Pattern – Payment Gateways

The **Adapter Pattern** converts the interface of a class into another interface the client expects. It enables incompatible interfaces to work together.

Implementation

```

// Target interface
interface PaymentProcessor {
    void processPayment(double amount);
}

// Adaptee classes (third-party gateways with different APIs)
class RazorpayGateway {
    public void makePayment(double amount) {
        System.out.println("Razorpay: Processing Rs. " + amount);
    }
}

class StripeGateway {
    public void charge(double amount) {
        System.out.println("Stripe: Charging $" + amount);
    }
}

// Adapters
class RazorpayAdapter implements PaymentProcessor {
    private RazorpayGateway gateway;

    public RazorpayAdapter(RazorpayGateway gw) { this.gateway = gw; }

    public void processPayment(double amount) {
        gateway.makePayment(amount);    // translate call
    }
}

```

```

    }

    class StripeAdapter implements PaymentProcessor {
        private StripeGateway gateway;
        public StripeAdapter(StripeGateway gw) { this.gateway = gw; }

        public void processPayment(double amount) {
            gateway.charge(amount);          // translate call
        }
    }

    // Test
    public class AdapterTest {
        public static void main(String[] args) {
            PaymentProcessor p1 = new RazorpayAdapter(new RazorpayGateway());
            PaymentProcessor p2 = new StripeAdapter(new StripeGateway());

            p1.processPayment(500.0);        // Razorpay: Processing Rs. 500.0
            p2.processPayment(20.0);         // Stripe: Charging $20.0
        }
    }
}

```

Key Points

- Client code only uses the *PaymentProcessor* interface – no gateway-specific code.
- Adding a new gateway needs only a new Adapter class.
- Useful when integrating legacy or third-party code without modification.

Exercise 5: Decorator Pattern – Notification System

The **Decorator Pattern** attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

Implementation

```

// Component interface
interface Notifier {
    void send(String message);
}

// Concrete component
class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

// Abstract decorator
abstract class NotifierDecorator implements Notifier {
    protected Notifier wrappee;
    public NotifierDecorator(Notifier n) { this.wrappee = n; }
    public void send(String message) { wrappee.send(message); }
}

// Concrete decorators

```

```

class SMSNotifierDecorator extends NotifierDecorator {
    public SMSNotifierDecorator(Notifier n) { super(n); }
    public void send(String message) {
        super.send(message);
        System.out.println("SMS: " + message);
    }
}

class SlackNotifierDecorator extends NotifierDecorator {
    public SlackNotifierDecorator(Notifier n) { super(n); }
    public void send(String message) {
        super.send(message);
        System.out.println("Slack: " + message);
    }
}

// Test
public class DecoratorTest {
    public static void main(String[] args) {
        // Send via Email + SMS + Slack
        Notifier notifier = new SlackNotifierDecorator(
            new SMSNotifierDecorator(
                new EmailNotifier()));
        notifier.send("Server is down!");
        // Output:
        // Email: Server is down!
        // SMS: Server is down!
        // Slack: Server is down!
    }
}

```

Key Points

- Decorators can be stacked in any combination at runtime.
- Follows **Open/Closed Principle** – add new channels without modifying existing code.
- More flexible than inheritance; avoids class explosion.

Exercise 6: Proxy Pattern – Image Viewer

The **Proxy Pattern** provides a surrogate or placeholder for another object to control access. Here it adds **lazy initialization** (load only when displayed) and **caching** (load once, reuse).

Implementation

```

// Subject interface
interface Image {
    void display();
}

// Real subject - expensive to create
class RealImage implements Image {
    private String filename;

    public RealImage(String filename) {
        this.filename = filename;
    }
}

```

```

        loadFromServer();
    }

    private void loadFromServer() {
        System.out.println("Loading image from server: " + filename);
    }

    public void display() {
        System.out.println("Displaying: " + filename);
    }
}

// Proxy - adds lazy loading + caching
class ProxyImage implements Image {
    private String filename;
    private RealImage realImage = null;    // cached instance

    public ProxyImage(String filename) {
        this.filename = filename;
    }

    public void display() {
        if (realImage == null) {           // lazy init
            realImage = new RealImage(filename);
        }
        realImage.display();               // use cached
    }
}

// Test
public class ProxyTest {
    public static void main(String[] args) {
        Image img = new ProxyImage("photo.jpg");

        // First call - loads from server
        img.display();    // Loading image... Displaying: photo.jpg

        // Second call - uses cache
        img.display();    // Displaying: photo.jpg (no reload)
    }
}

```

Key Points

- Image is not loaded until *display()* is first called – saves bandwidth.
- Subsequent calls reuse the cached *RealImage* – saves time.
- Client is unaware of the proxy; it works through the *Image* interface.

Exercise 7: Observer Pattern – Stock Market

The **Observer Pattern** defines a one-to-many dependency so that when one object (Subject) changes state, all dependents (Observers) are notified automatically.

Implementation


```

import java.util.ArrayList;
import java.util.List;

// Observer interface
interface Observer {
    void update(String stock, double price);
}

// Subject interface
interface Stock {
    void register(Observer o);
    void deregister(Observer o);
    void notifyObservers();
}

// Concrete subject
class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private String stockName;
    private double price;

    public StockMarket(String name) { this.stockName = name; }

    public void register(Observer o) { observers.add(o); }
    public void deregister(Observer o) { observers.remove(o); }

    public void setPrice(double price) {
        this.price = price;
        System.out.println(stockName + " price updated to: " + price);
        notifyObservers();
    }

    public void notifyObservers() {
        for (Observer o : observers) o.update(stockName, price);
    }
}

// Concrete observers
class MobileApp implements Observer {
    private String user;
    public MobileApp(String user) { this.user = user; }
    public void update(String stock, double price) {
        System.out.println("[Mobile-" + user + "] " + stock + " = " + price);
    }
}

class WebApp implements Observer {
    public void update(String stock, double price) {
        System.out.println("[WebApp] " + stock + " price changed to " + price);
    }
}

// Test
public class ObserverTest {
    public static void main(String[] args) {
        StockMarket tcs = new StockMarket("TCS");
        tcs.register(new MobileApp("Alice"));
        tcs.register(new WebApp());

        tcs.setPrice(3500.0);    // Both observers notified
    }
}

```

```

        tcs.setPrice(3550.5);    // Both observers notified again
    }
}

```

Key Points

- Loose coupling: Subject only knows the Observer interface, not concrete classes.
- New observers (e.g., EmailAlert) can be added without modifying StockMarket.
- Java's *java.util.Observable* and *EventListener* use this pattern.

Exercise 8: Strategy Pattern – Payment System

The **Strategy Pattern** defines a family of algorithms, encapsulates each one, and makes them interchangeable. The client can switch strategies at runtime.

Implementation

```

// Strategy interface
interface PaymentStrategy {
    void pay(double amount);
}

// Concrete strategies
class CreditCardPayment implements PaymentStrategy {
    private String cardNumber;
    public CreditCardPayment(String num) { this.cardNumber = num; }
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " via Credit Card " + cardNumber);
    }
}

class PayPalPayment implements PaymentStrategy {
    private String email;
    public PayPalPayment(String email) { this.email = email; }
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " via PayPal (" + email + ")");
    }
}

class UPIPayment implements PaymentStrategy {
    private String upiId;
    public UPIPayment(String id) { this.upiId = id; }
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " via UPI " + upiId);
    }
}

// Context
class PaymentContext {
    private PaymentStrategy strategy;

    public void setStrategy(PaymentStrategy s) { this.strategy = s; }

    public void executePayment(double amount) {
        strategy.pay(amount);
    }
}

```

```

    }
}

// Test
public class StrategyTest {
    public static void main(String[] args) {
        PaymentContext ctx = new PaymentContext();

        ctx.setStrategy(new CreditCardPayment("1234-5678"));
        ctx.executePayment(1500.0);

        ctx.setStrategy(new UPIPayment("alice@upi"));
        ctx.executePayment(750.0);
    }
}

```

Key Points

- Payment method can be swapped at runtime without changing client code.
- Eliminates conditional (if-else / switch) logic in the context class.
- Follows **Open/Closed Principle** – add new strategies without modifying context.

Exercise 9: Command Pattern – Home Automation

The **Command Pattern** encapsulates a request as an object, letting you parameterize clients with different requests, queue operations, and support undo.

Implementation

```

// Command interface
interface Command {
    void execute();
    void undo();    // optional - supports undo
}

// Receiver
class Light {
    private String location;
    public Light(String location) { this.location = location; }
    public void on() { System.out.println(location + " light ON."); }
    public void off() { System.out.println(location + " light OFF."); }
}

// Concrete commands
class LightOnCommand implements Command {
    private Light light;
    public LightOnCommand(Light l) { this.light = l; }
    public void execute() { light.on(); }
    public void undo() { light.off(); }
}

class LightOffCommand implements Command {
    private Light light;
    public LightOffCommand(Light l) { this.light = l; }
    public void execute() { light.off(); }
}

```

```

        public void undo()    { light.on(); }
    }

    // Invoker
    class RemoteControl {
        private Command command;
        private Command lastCommand;

        public void setCommand(Command c) { this.command = c; }
        public void pressButton() {
            command.execute();
            lastCommand = command;
        }
        public void pressUndo() {
            if (lastCommand != null) lastCommand.undo();
        }
    }

    // Test
    public class CommandTest {
        public static void main(String[] args) {
            Light livingRoom = new Light("Living Room");
            RemoteControl remote = new RemoteControl();

            remote.setCommand(new LightOnCommand(livingRoom));
            remote.pressButton();    // Living Room light ON.

            remote.setCommand(new LightOffCommand(livingRoom));
            remote.pressButton();    // Living Room light OFF.

            remote.pressUndo();      // Living Room light ON.  (undo)
        }
    }

```

Key Points

- Invoker (RemoteControl) is decoupled from Receiver (Light).
- Undo functionality is naturally supported by storing the last command.
- Commands can be queued, logged, or sent over a network.

Exercise 10: MVC Pattern – Student Records

The **MVC (Model-View-Controller) Pattern** separates application concerns: Model holds data, View displays it, Controller mediates between them.

Implementation

```

// Model
class Student {
    private String name;
    private String id;
    private String grade;

    public Student(String name, String id, String grade) {
        this.name = name;
    }
}

```

```

        this.id = id;
        this.grade = grade;
    }
    public String getName() { return name; }
    public String getId() { return id; }
    public String getGrade() { return grade; }
    public void setName(String n) { this.name = n; }
    public void setGrade(String g) { this.grade = g; }
}

// View
class StudentView {
    public void displayStudentDetails(String name, String id, String grade) {
        System.out.println("--- Student Details ---");
        System.out.println("Name : " + name);
        System.out.println("ID : " + id);
        System.out.println("Grade: " + grade);
    }
}

// Controller
class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String n) { model.setName(n); }
    public void setStudentGrade(String g) { model.setGrade(g); }

    public void updateView() {
        view.displayStudentDetails(
            model.getName(), model.getId(), model.getGrade());
    }
}

// Main
public class MVCTest {
    public static void main(String[] args) {
        Student student = new Student("Alzaheer", "CS2024", "A");
        StudentView view = new StudentView();
        StudentController c = new StudentController(student, view);

        c.updateView(); // display initial

        c.setStudentName("Alzaheer M");
        c.setStudentGrade("A+");
        c.updateView(); // display updated
    }
}

```

Key Points

- **Model** has no knowledge of View or Controller – pure data/logic.
- **View** only renders; it never modifies the model directly.

- **Controller** is the only component that bridges Model and View.
- Separation of concerns makes each component independently testable.

Exercise 11: Dependency Injection – Customer Management

Dependency Injection (DI) is a design principle where an object receives its dependencies from an external source rather than creating them itself. This promotes loose coupling and testability.

Implementation

```
// Repository interface
interface CustomerRepository {
    String findCustomerById(int id);
}

// Concrete repository
class CustomerRepositoryImpl implements CustomerRepository {
    public String findCustomerById(int id) {
        // Simulate DB lookup
        if (id == 1) return "Alice";
        if (id == 2) return "Bob";
        return "Unknown";
    }
}

// Mock repository (for testing - no DB needed)
class MockCustomerRepository implements CustomerRepository {
    public String findCustomerById(int id) {
        return "TestUser_" + id;
    }
}

// Service class - depends on CustomerRepository (not on the impl)
class CustomerService {
    private CustomerRepository repository;

    // Constructor Injection
    public CustomerService(CustomerRepository repo) {
        this.repository = repo;
    }

    public String getCustomerName(int id) {
        return repository.findCustomerById(id);
    }
}

// Main
public class DITest {
    public static void main(String[] args) {
        // Production wiring
        CustomerRepository repo    = new CustomerRepositoryImpl();
        CustomerService service    = new CustomerService(repo);
        System.out.println(service.getCustomerName(1)); // Alice

        // Test wiring (swap implementation - no code change in service)
```

```
CustomerRepository mockRepo = new MockCustomerRepository();  
CustomerService testService = new CustomerService(mockRepo);  
System.out.println(testService.getCustomerName(5)); // TestUser_5  
}  
}
```

Types of Dependency Injection

- **Constructor Injection** (used above) – dependencies passed via constructor. Preferred; makes dependencies explicit and supports immutability.
- **Setter Injection** – dependencies set via setter methods after construction. Useful for optional dependencies.
- **Field/Interface Injection** – used by frameworks (Spring, Jakarta EE) via annotations like *@Autowired*.

Key Points

- CustomerService depends on the *interface*, not the concrete class.
- Swapping implementations (production vs mock) requires zero changes to the service.
- **SOLID Principle:** Dependency Inversion – high-level modules depend on abstractions.